Doomscroll

System Architecture — Lab 3

Version 1.0
Prepared by: Project Team -- SC2006
Date: 4 October 2026

1. Overview

Doomscroll is a single-process desktop application with no client/server split and no persistent background service: it runs entirely inside one Python process started by app.py, for the duration the user keeps a window open, on the user's own machine. The architecture below reflects the actual module layout under src/doomscroll/, not an idealised redesign.

2. Layered View

The codebase separates into four layers, from the user-facing launcher down to the operating-system integrations each platform needs:

- Launcher (app.py, src/doomscroll/ui/): the Tkinter Main Page and Settings Page. Starts every other flow as a subprocess and tails its combined stdout/stderr into the Settings page's Log.
- Flow scripts (scripts/*.py): one script per user-facing flow -- Quick Start calibration (scripts/calibrate.py), Record/train/calibrate (scripts/train_model.py and related), Calibrate only, and the live Autoscroll session (scripts/phase2_gaze_scroll.py). Each is launched as its own OS subprocess so a crash in one flow cannot take down the launcher.
- Tracking/decision pipeline (src/doomscroll/gaze/, src/doomscroll/ml/, src/doomscroll/fixation/, src/doomscroll/content/gaze_content.py): head pose and gaze estimation (GazeCalibrator for the Math Model, TrainedGazeModel for the CNN Model), the Attention Gate, the Content Gaze Tracker, and the Scroll Trigger -- the logic that decides, frame by frame, whether to scroll.
- Platform backends (src/doomscroll/scroll/, src/doomscroll/windowing/, src/doomscroll/content/): the only code that differs by operating system -- posting a scroll event, window-bounds lookup, and (macOS only) Vision OCR screen scanning.

3. Design Patterns Actually Used

One design pattern is genuinely present in the codebase, rather than named retroactively for this report. Gaze-model selection, by contrast, is not an instance of it, and is described honestly as what it is.

3.1 Strategy, via a per-platform factory — the OS integration layer

Three small areas of OS-specific behaviour are each defined as an abstract base class with exactly the methods the rest of the system needs, plus a factory function that picks the concrete implementation once, by sys.platform:

- ScrollInjector (src/doomscroll/scroll/base.py) -- one scroll() method. get_scroll_injector() (src/doomscroll/scroll/__init__.py) returns WindowsScrollInjector or MacOSScrollInjector.
- WindowBoundsProvider (src/doomscroll/windowing/base.py) -- one get_focused_window_bounds() method. get_window_bounds_provider() returns WindowsWindowBoundsProvider or MacOSWindowBoundsProvider.
- ContentScanner (src/doomscroll/content/base.py) -- one scan() method. get_content_scanner() returns MacOSVisionContentScanner on macOS, and raises NotImplementedError on any other platform -- there is no Windows OCR backend at all, which is how Use Case 4.1's platform fallback (tier 1 only on Windows) is actually realised: not a capability check scattered through the Scroll Trigger, but the simple absence of an implementation to construct.

This is the one place the codebase deliberately keeps call sites (scripts/phase2_gaze_scroll.py, the Scroll Trigger, the Attention Gate) unaware of which operating system they are running on.

3.2 Gaze-model selection is a plain branch, not Strategy

GazeCalibrator (Math Model) and TrainedGazeModel (CNN Model) do not share a base class or a common method name (GazeCalibrator exposes predict(); TrainedGazeModel exposes predict_frac() / predict_px()). scripts/phase2_gaze_scroll.py selects between them with a direct `if args.model == "cnn": ... elif args.model == "math": ...` branch at startup, not a polymorphic call through a shared interface. Both pipelines do feed the same downstream Scroll Trigger and Attention Gate, but the model choice itself is a simple conditional, not a design pattern.

No other named pattern (Observer, Singleton, Command, Builder, etc.) is deliberately structured into the codebase; the launcher's subprocess-and-log-tailing design and the tracking pipeline's straight-line per-frame update() calls do not need one.

4. Persistent Data

The system has no database and no network-hosted storage. Everything it persists lives on the local filesystem, under the operating system's standard per-application data directory (resolved once per machine via the platformdirs package):

- Calibration data -- the Math Model's Calibration Profile and the CNN Model's per-user Affine Correction, plus the pooled calibration-session records used by Quick Start's optional pooling -- stored as joblib-serialised files (src/doomscroll/gaze/session_store.py, src/doomscroll/gaze/calibration.py).

- The Training Dataset (face crops and numeric feature vectors from past Recording Sessions) and the trained CNN Model weights (models/gaze_cnn.pt) -- joblib/PyTorch files, retrained in place after every new Recording Session (src/doomscroll/ml/model.py).
- A per-machine Participant ID (Requirement 3.6.5) -- a plain text file (device_participant_id.txt); each Recording Session's own metadata is a small JSON file alongside it (src/doomscroll/recording/session.py).

None of this is encrypted at rest and none of it leaves the machine; Requirement 4.3.1 and 4.3.2 (Privacy and Data Protection) describe the scope of what is, and is not, kept.

5. Access Control

Doomscroll has no user accounts, no login, and no multi-user data separation -- it is a single-user local desktop application, and every file it reads or writes belongs to the one operating-system user account running it. Access control, in the sense of authenticating or authorising distinct users against shared resources, does not apply to this system. The only access-control-adjacent behaviour is the operating system's own camera-permission prompt (Use Case 1.1, Requirement 3.1.1), which is standard OS policy rather than anything this application implements itself.